

Provably Correct Post-Update Incremental Maintenance of RDB2RDF Views

Vânia Maria Ponte Vidal
Federal University of Ceará
Fortaleza, Ceará, Brazil
vvidal@lia.ufc.br

José Renato da Silva Freitas
Federal University of Ceará
Fortaleza, Ceará, Brazil
jrenatosfreitas@gmail.com

Narciso Arruda
Federal University of Ceará
Fortaleza, Ceará, Brazil
narcisoarruda@gmail.com

Marco A. Casanova
Pontifical Catholic University of Rio de Janeiro
Rio de Janeiro, Rio de Janeiro, Brazil
casanova@inf.puc-rio.br

Abstract

A strategy for exposing relational data as part of a knowledge graph is to define RDB2RDF views that map relational tuples to RDF triples using transformation rules. These views are often materialized in RDF stores to improve query performance and data availability, a process that requires incremental mechanisms to maintain the views in the presence of updates to the underlying relational database. A key problem in the incremental maintenance of RDB2RDF views is computing the changesets to apply to a view, which traditionally depend on the pre-update state of the relational database. The central contribution of this paper is a formal model of transformation rules that yields provably correct changesets operating directly on the post-update relational database state. The second contribution is to propose an operational architecture for incremental maintenance of RDB2RDF views that realizes the formal model. The architecture presupposes that the view is defined by R2RML statements, features AFTER triggers, and includes a maintenance worker to implement transformation rules generated from the R2RML statements. The third contribution is a feasibility study of the operational architecture, based on an RDB2RDF view defined by R2RML statements, for the MusicBrainz database. The architecture and the feasibility study provide evidence that the formal model is sufficiently constructive to be translated into executable maintenance code.

Keywords

LLM SQL Code Generation, Semantic View, Enterprise Knowledge Graph, Formal Specification

1 Introduction

A strategy for exposing relational data as part of a knowledge graph is the definition of *RDB2RDF views*, which map relational tuples into RDF triples using transformation rules. These views are often materialized in RDF stores to improve query performance and data availability. However, when the relational source is frequently updated, the materialized RDB2RDF view must be continuously synchronized with the underlying database. Re-computing the entire RDB2RDF view after each update is usually impractical, motivating the need for *incremental maintenance techniques* that compute only the changes to the view.

A well-established approach for incremental view maintenance is to compute *changesets*, represented as a pair $\langle \Delta^-(u), \Delta^+(u) \rangle$, capturing the triples removed from and added to the RDB2RDF

view after a relational update u . Prior work [19] has shown that, for entity-preserving RDB2RDF statements, it is sufficient to identify the *pivot tuples* whose RDF contributions may be affected by the update and recompute their associated RDF state.

The formal framework introduced in [19] provides a rigorous method for computing such changesets based on transformation rules over relational schemas. In particular, in this framework, the deletion component $\Delta^-(u)$ is defined with respect to the pre-update database state. Conceptually, this definition is correct, as it captures precisely the RDF triples that must be removed from the view. However, it introduces a practical limitation when implementing the framework with database triggers, namely, relying on the pre-update database state.

The central contribution of this paper is to challenge this assumption by introducing a formal model that yields provably correct changesets operating directly on the post-update state. It shows that, for a transformation rule without multiple occurrences of the same relation, both deletion and insertion components can be derived using only the post-update state, the transition tables, and the pre-update view state. For a transformation rule with multiple occurrences of the same relation R , it captures the impact of deletions and insertions on R directly in the evaluation of the transformation rule, still without using the pre-update database state.

The second contribution of the paper is to propose an operational architecture for the incremental maintenance of RDB2RDF views that realizes the formal model. The architecture decouples SPARQL computation from relational transactions, thereby avoiding the execution of SPARQL queries inside relational transactions and isolating the triplestore. The architecture presupposes that the RDB2RDF view is defined by R2RML statements, which requires a preprocessing step to translate such statements into transformation rules. The paper proposes implementing this preprocessing step using an LLM prompted with the formal model. Then, the paper discusses how to translate the transformation rules into AFTER triggers, again using an LLM prompted with the formal model.

The third contribution of the paper is a feasibility study of the operational architecture. The study evaluates the complete implementation cycle for an RDB2RDF view built on a version of the MusicBrainz database. The database features a schema with 374 relations that encompass information about artists, release groups, etc. The RDB2RDF view uses the *Music Ontology* vocabulary and is defined by 57 R2RML statements.

To summarize, existing incremental RDB2RDF maintenance formulations are difficult to realize using statement-level AFTER triggers because their deletion changesets depend on the

pre-update relational state, which is no longer directly available when such triggers execute. The proposed formal model removes this mismatch by reformulating the computation of changesets in terms of the post-update relational state, the transition tables, and the pre-update RDF view. Equally important, the resulting changesets are provably correct. The formal model is also sufficiently constructive to support specification-guided generation of executable maintenance triggers from R2RML mappings.

Lastly, it should be stressed that an LLM is not used to establish the correctness of the changesets. Rather, the LLM-based study provides operational evidence that the formal model is sufficiently constructive to be translated into executable maintenance code.

The paper is organized as follows. Section 2 covers related work. Section 3 defines the notion of entity-preserving RDB2RDF views. Section 4 summarizes the theory of incremental RDB2RDF view maintenance. Section 5 discusses the operational architecture. Section 6 describes the feasibility study conducted and discusses the results. Finally, Section 7 contains the conclusions.

2 Related Work

2.1 Brief Review of Previous Work on IVM

The Incremental View Maintenance (IVM) problem has been studied in the literature for a long time [14]. Early work covered relational views [7], object-oriented views [2], semi-structured views [25], and XML views [11], just to name a few efforts.

Some more recent approaches include the following. As for generic IVM tools, DBSP [6] introduces an expressive language that enables efficient IVM for queries written in multiple languages, including SQL and Datalog. Enzyme [23], developed at Databricks, is an IVM engine that uses materialized views as building blocks. OpenIVM [4] compiles view definitions into SQL to propagate changes to a view's table. `pg_ivm`¹ is a production-ready implementation of IVM for PostgreSQL.

As for IVM SPARQL and graph databases, Incremunica [17] integrates multiple incremental operators, allowing it to adapt to different queries and data for optimal performance, and supporting federated querying, dynamically adding and removing sources during query execution, SPARQL Query Language support, multiple IVM techniques, and client-side execution. The counting algorithm, a classic approach to IVM for relational queries, is adapted in [15] to SPARQL queries over RDF datasets. Views over property graphs, both virtual and materialized, are considered in [9] for integrating heterogeneous data using rewriting techniques.

Solutions in specific settings include the following. The maintenance of self-join-free conjunctive queries over a dynamic database, where tuples can be inserted or deleted, is investigated in [10]. A maintenance approach that combines delta queries, trees of materialized views, and heavy-light data partitioning is introduced in [1], whose update time matches or improves the best update time reported in prior work. The incorporation of knowledge about foreign keys into IVM to speed up performance is investigated in [16], which reports experiments in Redshift showing that the proposed technique improved the execution times of the whole refresh process up to 2 times, and up to 2.7 times the process of calculating the necessary changes that will be applied to the materialized view. Finally, a framework for maintaining sketches incrementally under updates is introduced in [12].

¹https://github.com/sraoss/pg_ivm

2.2 RDB2RDF versus Relational IVM

RDB2RDF IVM differs from relational IVM in at least three aspects.

First, there are the (obvious) questions of how and why the views are defined. A relational view is defined by a relational expression to hide schema details or simplify query processing, among other reasons, whereas an RDB2RDF view is intended to publish relational data as an RDF graph, while preserving the identity of the represented entities. Following [19], this paper defines an RDB2RDF view by a set of transformation rules (TRs), whose bodies include select-projection-join (SPJ) relational expressions; the results of the TRs jointly contribute to the RDF representation of the same or related entities.

Second, there is the question of duplicated derivations. Classical relational IVM computes changes to a materialized view by propagating base-relation insertions and deletions through the relational expression defining the view. For views defined by SPJ relational expressions, delta expressions are typically obtained by replacing occurrences of updated base relations with their corresponding deltas and evaluating the resulting expressions over appropriate pre- and post-update relation states. Classical approaches handle duplicate derivations through bag semantics or by maintaining derivation counts for materialized tuples [7, 14]. The PostgreSQL extension `pg_ivm` follows this general principle, using statement-level AFTER triggers and transition tables to capture inserted and deleted tuples.

By contrast, RDB2RDF IVM depends on RDF graphs, which have a set semantics: the same triple may be supported by multiple derivations involving different TRs. Consequently, when the derivation of an RDF triple t is no longer valid, it does not necessarily imply the deletion of t from the RDF view.

Third, there is the IVM level. Relational IVM deals with relational delta tuples. However, when RDB2RDF views are defined by TRs, IVM must account for the combined RDF contributions of the TRs at the entity level. This aspect motivated shifting, in this paper, the unit of IVM from relational delta tuples to the RDF contributions of the affected entities.

2.3 Contributions to RDB2RDF IVM

This paper builds upon the formal model proposed in [19, 20], reviewed and extended in Section 3. However, such previous approaches depended on accessing the database state before the update to generate the view changesets.

By contrast, the RDB2RDF IVM method proposed in this paper enables a post-update formulation that does not require reconstruction of the exact pre-update relational state, as discussed in Section 4. The required RDB2RDF changes can instead be derived from the post-update database state, the update information, and the pre-update RDF view state.

Also, by exploiting the entity-preserving property of the TRs, the RDB2RDF IVM method proposed in this paper identifies the TRs relevant to an update, determines the affected pivot entities, and recomputes only their RDF contributions. This process is covered in Section 4.2 and condensed in Algorithm 1. It handles multiple derivations of the same RDF triple by determining whether the triple remains supported after the update, rather than deriving RDF insertions and deletions directly from relational tuple multiplicities.

2.4 On the Use of LLMs

Large Language models (LLMs) have demonstrated remarkable performance across a wide range of Natural Language (NL) tasks and beyond. In formal software engineering (SE), LLMs have been employed, for example, to address formal requirements engineering [8], generate verifiable specifications from informal requirements [5], help with conceptual modeling [13], facilitate software testing [3, 22], and build trustworthy AI agents [24]. This paper falls in this broad area.

The definition of the formal model outlined in Sections 3 and 4 co-evolved with the prompts used in Sections 5 and 6 as follows. During the development of the formal model, attempts to generate executable maintenance logic from the definitions exposed implicit assumptions and formulations that, although meaningful, were not sufficiently constructive for direct implementation. The resulting feedback led to clearer definitions and a more explicit connection between the formal changeset semantics and the data available to statement-level AFTER triggers.

After this exploratory stage, the paper's development focused on demonstrating that LLMs can process a (complex and lengthy) formal model and produce the formally specified database code. Specifically, Section 5.3 first shows how to translate R2RML statements into transformation rules using an LLM prompted with the formal model. Then, it demonstrates how to translate the transformation rules into the required AFTER triggers, again using an LLM prompted with the formal model.

The feasibility study described in Section 6 then investigates the operationalization of the formal model proposed in Section 5.

3 Formal Specification of Entity-Preserving RDB2RDF Views

3.1 Entity-Preserving RDB2RDF Views

An *RDB2RDF view* is a triple $\mathcal{W} = (V, \mathcal{S}, M)$, where V is the vocabulary of the *target ontology*, $\mathcal{S}$ is the *source relational schema*, and M is a set of *mappings* between V and $\mathcal{S}$, defined by *transformation rules* (TRs).

The semantics of an RDB2RDF view is defined in Section 3.3. A view $\mathcal{W} = (V, \mathcal{S}, M)$ satisfies the *entity-preserving property* iff:

- the instances of the classes in V correspond to tuples of selected relations of $\mathcal{S}$, so that distinct tuples from the same relation generate distinct class instances. These tuples and relations are called *pivot tuples* and *pivot relations*;
- the values of datatype properties in V of these instances are given by (functions of) attribute values of the corresponding tuples, or of related tuples;
- the object properties in V correspond to relationships between tuples of the pivot relations of $\mathcal{S}$.

A *transformation rule* (TR) of $\mathcal{W}$ is an expression of the form $C(x) \leftarrow Q(x)$ or of the form $P(x,y) \leftarrow Q(x,y)$, where C and P are class and property names in V , and $Q(x)$ and $Q(x,y)$ are queries over $\mathcal{S}$ whose target clauses contain one and two variables, respectively.

The definitions that follow extend those first introduced in [19] to include new types of TRs. The problem of generating TRs is addressed in [20, 21] and is outside of the scope of this work. Section 3.2 contains examples of TRs for the RDB2RDF view used in the feasibility study.

Table 1 lists a set of TR patterns that lead to the definition of relational to RDF statements that guarantee that the RDB2RDF view satisfies the entity-preserving property by construction.

Table 1 defines three types of TR patterns: *Class Transformation Rule* (CTR), *Datatype Property Transformation Rule* (DTR), and *Object Property Transformation Rule* (OTR).

Intuitively, a CTR Ψ maps tuples of a relation R into RDF instances of a class C . It establishes a semantic correspondence between a pivot tuple r and an RDF resource x , such that each pivot tuple is associated with at most one RDF instance, and distinct pivot tuples generate distinct RDF resources. Thus, the mapping preserves the identity of relational entities in the RDB2RDF view.

There are two types of DTRs. A *Local DTR* has no relational path in the body so that the datatype values are obtained directly from attribute values of the pivot tuples.

A *Path DTR* has a relational path in the body, and the datatype values are obtained from tuples reachable through the relational path from the pivot tuples.

There are also two types of OTRs. A *regular OTR* Ψ defines object properties by relating an RDF instance generated from a pivot tuple t to an RDF instance generated from a pivot tuple u reached from t through a relational path in the source schema. Hence, OTRs capture relationships between entities represented in the database.

A *reification OTR* relates an RDF instance generated from a pivot tuple t to a URI² obtained by reification of a literal l created from attribute values of a pivot tuple u reached from t through a relational path in the source schema.

By convention, the *pivot relation* of a TR Ψ , denoted $\text{pivot}(\Psi)$, is always the first relation that appears in the body of Ψ .

The transformation rules in Table 1 and in the feasibility study use several predicates. The simplest ones are:

- $\text{nonNull}(v)$ holds iff value v is not null.
- $\text{RDFLiteral}(u, A, R, v)$ holds iff v is the literal representation of a value u of attribute A of R .
- $\text{RDFIRI}(u, o)$ holds iff the non-null value u is converted into the RDF IRI o (that is, o is just the literal u treated as a URI).
- $F(r, s)$ holds iff a tuple r of R is related to tuple s of S by a foreign key of the form $F(R:L, S:K)$.

The more complex predicates are as follows.

A *relational path* φ is a list of the form " $[F_1, \dots, F_n]$ ", where F_j is either the name of a foreign key, indicating that the path traverses F_j from R_j and R_{j+1} , or of the form $(F_j)^{-1}$, indicating that the path traverses F_j from R_{j+1} to R_j , in the reverse direction. The expression $\text{path}(\Psi)$ denotes the relational path of Ψ . A relational path φ induces a *predicate path* $\varphi(x, y)$, which is evaluated in a database state by joining the relations in the path in the directions indicated.

Let R be the pivot relation of a transformation rule Ψ , and r be a pivot tuple in R .

A *URI template* for R is a string pattern p with place holders $\langle h_1, \dots, h_n \rangle$ such that, when the place holders are appropriately instantiated, the resulting string becomes a URI.

A URI s is *associated with* r via p iff the predicate

$$\text{hasURI}(p, \langle e_1, \dots, e_n \rangle, s)$$

holds, where

- $\langle e_1, \dots, e_n \rangle$ is an ordered sequence of expressions, where each e_i is of the form:

²Throughout this paper, the term URI is used for consistency with the URI-constructor notation. Formally, every URI occurring in an RDF triple denotes an RDF IRI.

Table 1: Transformation Rule Patterns

TR	Transformation Rule Pattern
CTR	$\psi : C(s) \leftarrow R(r), URI_R(r, s), \delta[r]$ where - ψ is the name of the transformation rule - R is the name of a relation or a view in the source schema $\mathbf{S}$ - r is a tuple variable bound to tuples of R - s is the URI associated with the tuple bound to r - $URI_R(r, s)$ is a URI constructor predicate as defined in the text - $\delta[r]$ is an optional selection over R , among other relations
Local DTR	$\psi : P(s, v) \leftarrow R_D(r), B_D[r, s], nonNull(r.A_1), \dots, nonNull(r.A_k),$ $RDFLiteral(r.A_1, "A'_1", "G'', v_1), \dots, RDFLiteral(r.A_k, "A'_k", "G'', v_k), T([v_1, \dots, v_k], v), \delta[r]$ where - ψ is the name of the transformation rule - P is a datatype property of V with domain D - $nonNull$ and $RDFLiteral$ are predicates as defined in the text - " $R_D(r), B_D[r, s]$ " is the RHS of the CTR ψ_D that matches class D with pivot relation R_D of ψ_D - $A_1, \dots, A_k$ are attributes of R_D - T is an optional function that transforms values of attributes $A_1, \dots, A_k$ of R_D to values of property P - $\delta[r]$ is an optional selection over R_D , among other relations
Path DTR	$\psi : P(s, v) \leftarrow R_D(r), B_D[r, s], nonNull(r.A_1), \dots, nonNull(r.A_k),$ $RDFLiteral(r.A_1, "A'_1", "G'', v_1), \dots, RDFLiteral(r.A_k, "A'_k", "G'', v_k), T([v_1, \dots, v_k], v),$ $\varphi(r, g), \delta[r, g]$ where - $\varphi(r, g)$ is a predicate path starting on R_D and ending on a relation R_G - $\delta[r, g]$ is an optional selection over R_D and R_G , among other relations - the other terms are as in a Local DTR
Regular OTR	$\psi : P(s, o) \leftarrow R_D(r), B_D[r, s], R_G(g), B_G[g, o], \varphi(r, g), \delta[r, g]$ where - " $R_G(g), B_G[g, o]$ " is the RHS of the CTR ψ_G that matches class G with pivot relation R_G of ψ_G - the other terms are as in a Path DTR
Reification OTR	$\psi : P(s, o) \leftarrow R_D(r), B_D[r, s], nonNull(r.A_1), \dots, nonNull(r.A_k),$ $RDFLiteral(r.A_1, "A'_1", "G'', v_1), \dots, RDFLiteral(r.A_k, "A'_k", "G'', v_k), T([v_1, \dots, v_k], v),$ $RDFIRI(u, o), \delta[r]$ where - $RDFIRI(u, o)$ is as defined in the text - the other terms are as in a Local DTR

- $e_i = r.A$, where $r.A$ denotes an attribute of the pivot tuple.
- $e_i = [FK(r, t)/t.A]$, where $[FK(r, t)/t.A]$ denotes the value of attribute A obtained after following the foreign key FK from the pivot tuple r to tuple t such that the foreign-key navigation must be functional from the pivot relation to the referenced relation.
- s is the result of instantiating each h_i by e_i in p .

Finally, to simplify the notation, a second predicate, called a *URI constructor*, is introduced by definition:

$$URI_R(r, s) \equiv hasURI(p, \langle e_1, \dots, e_n \rangle, s)$$

Figure 1(a) shows the *MusicBrainz* schema, Figure 1(b) illustrates a fragment of the *Music Ontology*, and Table 2 lists some of the transformation rules compiled from the R2RML statements.

Table 2 illustrates URI definitions and the types of TRs:

URI definition. The first row indicates that the URIs associated with the pivot tuples of relation *artist* are defined by

$$hasURI("http://musicbrainz.org/artist/\langle h \rangle", \langle r.gid \rangle, s)$$

3.2 Examples of Transformation Rules

The case study of Section 6 uses an RDB2RDF view defined on top of the *MusicBrainz* database. The view adopts the *Music Ontology* vocabulary and is defined using R2RML statements, which are automatically translated into transformation rules (see Section 5.3).

That is, the URI s associated with the pivot tuple r is generated by replacing h by " $r.gid$ " in the template " $http://musicbrainz.org/artist/\langle h \rangle$ ".

Class Transformation Rule (CTR). Ψ_1 is a CTR that defines the instances of class *mo:MusicArtist*.

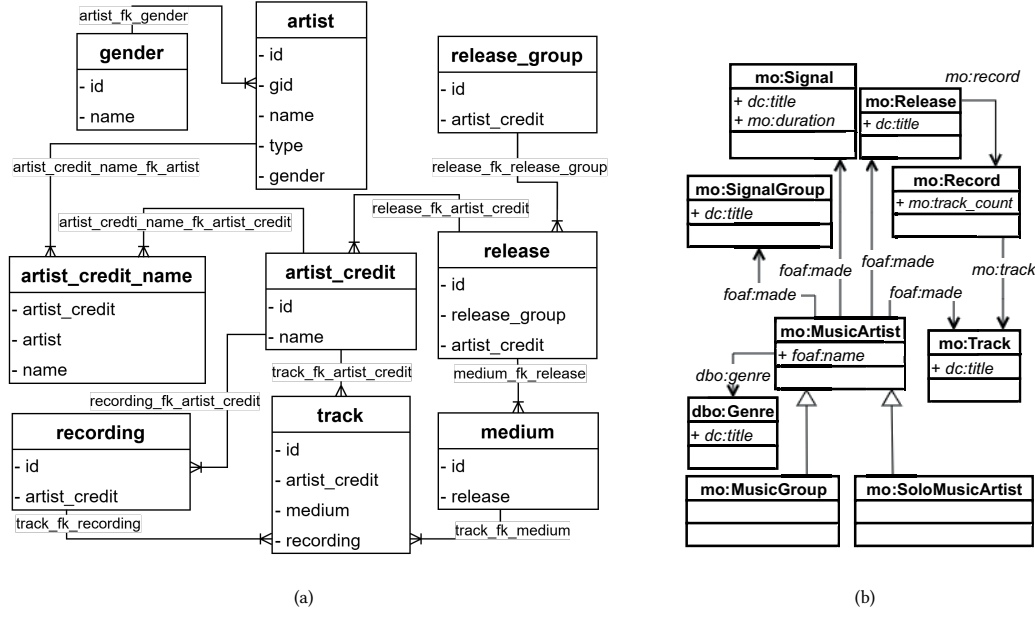


Figure 1: (a) MusicBrainz Schema and (b) Fragment of LinkedBrainz View Ontology.

Table 2: Examples of transformation rules.

TR	Transformation Rules
	$URI_artist(r, s) \equiv hasURI("http://musicbrainz.org/artist/<h>", \langle r.gid \rangle, s)$
Ψ_1	$mo:MusicArtist(s) \leftarrow artist(r), URI_artist(r, s)$
Ψ_6	$foaf:made(s, g) \leftarrow artist(r), URI_artist(r, s), track(t), URI_track(t, g), [(artist_credit_name_fk_artist)^{-1}, artist_credit_name_fk_artist_credit, (track_fk_artist_credit)^{-1}](r, t)$
Ψ_8	$foaf:name(s, v) \leftarrow artist(r), URI_artist(r, s), nonNull(r.name), RDFLiteral(r.name, "name", "artist", v)$
Ψ_9	$foaf:gender(s, v) \leftarrow artist(r), URI_artist(r, s), gender(g), nonNull(g.name), lower(g.name, u) RDFLiteral(u, "name", "gender", v), [artist_fk_gender](r, g),$

Object Property Transformation Rule (OTR). Ψ_6 is an OTR. The body of Ψ_6 has a predicate path

$$[(artist_credit_name_fk_artist)^{-1}, artist_credit_name_fk_artist_credit, (track_fk_artist_credit)^{-1}](r, t)$$

with three foreign keys (defined in Figure 1(a)) that connect tuples r of relation *artist* with tuples t of relation *track*; the first and the third foreign keys should be traversed in the reverse direction.

Datatype Property Transformation Rule (DTR). Ψ_8 is a Local DTR that defines the datatype property *foaf:name* as the name of an artist, whereas Ψ_9 is a Path DTR that defines the datatype property *foaf:gender* as the artistic gender of an artist, using table *gender*.

3.3 Hierarchical RDF-State Semantics of RDB2RDF Views

Let $\mathcal{W} = (V, S, M)$ be an RDB2RDF view and σ be a database state.

The *state* of $\mathcal{W}$ in σ is the set of quads (s, p, o, g) generated by the TRs in M when applied to σ , where the fourth element g is the URI of the TR that generated the triple (s, p, o) . Intuitively, each

TR Ψ induces a named graph containing exactly the triples generated by Ψ , which provides a simple and operational mechanism for provenance tracking and avoids ambiguity in the presence of duplicate triples generated by different TRs. The state of $\mathcal{W}$ is defined compositionally in three hierarchical levels.

Let $\Psi \in M$ be a TR with tuple variables $r_1, \dots, r_n$ associated with relation schemes $R_1, \dots, R_n$. For tuples $p_1 \in R_1(\sigma), \dots, p_n \in R_n(\sigma)$, the *application of Ψ under σ* is the set of RDF triples produced by evaluating Ψ with bindings $r_i = p_i$, for $i = 1, \dots, n$, over σ , denoted

$$\Psi[r_1 \leftarrow p_1, \dots, r_n \leftarrow p_n](\sigma)$$

Assume that the pivot relation of Ψ is R^* . For a pivot tuple $t \in R^*(\sigma)$, the *RDF contribution of t under Ψ and σ* is the set:

$$RDF_State[\Psi](t, \sigma) = \{(s, p, o, g) \mid (s, p, o) \in \Psi[r \leftarrow t](\sigma), g = URI(\Psi)\}$$

Finally, the *RDF state of the view $\mathcal{W}$ under σ* is the set

$$RDF_State(\mathcal{W}, \sigma) = \bigcup_{\Psi \in M} \bigcup_{t \in pivot(\Psi)(\sigma)} RDF_State[\Psi](t, \sigma)$$

That is, the state of an RDB2RDF view is obtained by aggregating the RDF contributions generated by all TRs for all pivot tuples in the database state.

4 Post-Update Incremental Maintenance of RDB2RDF Views

4.1 The Incremental Maintenance Problem

An *update* on a relation R is a pair $u = (D, I)$, where D and I are (possibly empty) sets of tuples, indicating that the tuples in D must be first deleted from R and then the tuples in I must be inserted into R . For simplicity, $D \cap I = \emptyset$, by assumption. The application of u induces a transition from a *pre-update* database state σ_0 to the *post-update* state σ_1 .

Let u be an update and σ_0 and σ_1 be the pre-update and post-update database states of u . Let $W_0 = \text{RDF_State}(W, \sigma_0)$ and $W_1 = \text{RDF_State}(W, \sigma_1)$ be the view states before and after the update u .

A *view changeset* is a pair $\langle \Delta^-(u), \Delta^+(u) \rangle$, where $\Delta^-(u)$ is the set of quads to be removed from W_0 , called the *deletion changeset*, and $\Delta^+(u)$ is the set of quads to be inserted into W_0 , after deleting the quads in $\Delta^-(u)$, called the *insertion changeset*.

The view changeset is *correct* iff

$$W_1 = (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$$

In other words, the changeset $\langle \Delta^-(u), \Delta^+(u) \rangle$ is correct if applying it to the pre-update view state W_0 yields the same result as recomputing the view over σ_1 .

An approach for computing changesets is to use database triggers. However, in relational systems, transition tables such as `deleted_R` and `inserted_R` are only available in *AFTER statement-level triggers*, when the database is already in the post-update state σ_1 . As a consequence, the pre-update state σ_0 is no longer directly accessible. A key limitation of previous approaches is that they often require reconstructing portions of the pre-update state of the relation R being updated to compute $\Delta^-(u)$. Section 4.2 then introduces a strategy for computing $\Delta^-(u)$ and $\Delta^+(u)$ that avoids this problem.

4.2 A Post-Update Formulation of Changeset Computation

This section presents Algorithm 1 that computes insertion and deletion changesets $\Delta^-(u)$ and $\Delta^+(u)$. Algorithm 1 organizes the definitions in [18] into pseudocode, following a compositional structure, and ensures that all affected RDF quads are computed using only the post-update database state, the tuples recorded in the transition tables, and the pre-update RDB2RDF view state.

4.2.1 Overview

Let $\mathcal{W} = (V, \mathcal{S}, M)$ be an RDB2RDF view and Ψ be a transformation rule (TR) in M . Let $u = (D, I)$ be an update over a relation R , and σ_1 be the database state after applying u .

Algorithm 1 proceeds by decomposing the computation of the changeset into TR-level contributions. The first step is to identify the TRs that are relevant to updates on R :

- (1) Ψ is *pivot-relevant* to updates on R iff $\text{pivot}(\Psi) = R$, that is, R is the first relation in the body of Ψ .
- (2) Ψ is *relation-relevant* to updates on R iff R occurs in the body of Ψ in an occurrence other than the pivot occurrence.

- (3) Ψ is *relevant* to updates on R iff Ψ is pivot-relevant or relation-relevant to updates on R ; $\text{Relev}(R)$ denotes the set of transformation rules relevant to updates on R .

Note that Ψ may be both pivot-relevant and relation-relevant to R . If Ψ is pivot-relevant to updates on R , the tuples in D and I are themselves pivot tuples of Ψ , and their RDF contributions are directly affected by the update. If Ψ is relation-relevant to updates on R , the update affects the evaluation of Ψ indirectly: tuples in the pivot relation R^* of Ψ whose derivations depend on tuples in D or I may have their RDF contributions modified.

For example, consider the TR Ψ_6 in Table 2. Then, Ψ_6 is relevant to *artist*, which is the pivot relation of Ψ_6 , to *track*, which explicitly occurs in $\text{path}(\Psi_6)$, and to *artist_credit_name* and *artist_credit*, which are used in foreign keys that occur in $\text{path}(\Psi_6)$.

Next, for each rule $\Psi \in \text{Relev}(R)$, the algorithm computes the contribution of Ψ to the changesets by deriving the deletion and insertion components, denoted $\Delta^-[\Psi](u)$ and $\Delta^+[\Psi](u)$, respectively, which capture the RDF quads that must be removed from and added to the view, using Ψ , with respect to u .

The overall changeset is then obtained by aggregating the contributions of all relevant rules:

$$\Delta^-(u) = \bigcup_{\Psi \in \text{Relev}(R)} \Delta^-[\Psi](u) \quad \Delta^+(u) = \bigcup_{\Psi \in \text{Relev}(R)} \Delta^+[\Psi](u)$$

The key idea is that changesets can be computed by characterizing the effect of an update on each relevant TR contribution.

The deletion and insertion components, $\Delta^-[\Psi](u)$ and $\Delta^+[\Psi](u)$, in turn have two components:

$$\Delta^-[\Psi](u) = \Delta_{\text{pivot}}^-[\Psi](u) \cup \Delta_{\text{rel}}^-[\Psi](u)$$

$$\Delta^+[\Psi](u) = \Delta_{\text{pivot}}^+[\Psi](u) \cup \Delta_{\text{rel}}^+[\Psi](u)$$

where:

- $\Delta_{\text{pivot}}^+[\Psi](u)$ denotes the contribution from the pivot-relevant effect of Ψ .
- $\Delta_{\text{rel}}^+[\Psi](u)$ denotes the contribution from the relation-relevant effect of Ψ .

4.2.2 Major Steps of the Algorithm

This section briefly explains the major steps of Algorithm 1.

Step 1: Computing the pivot-relevant contribution (Lines 9–18).

If Ψ is pivot-relevant to R , then the tuples in D and I are themselves pivot tuples of Ψ .

In this case, the deletion contribution $\Delta_{\text{pivot}}^-[\Psi](u)$ consists of the quads in the pre-update RDF state W_0 generated by Ψ from deleted pivot tuples $d \in D$. These quads become invalid after the update and must be removed from the view.

Symmetrically, the insertion contribution $\Delta_{\text{pivot}}^+[\Psi](u)$ consists of the quads generated by applying Ψ to inserted pivot tuples $i \in I$, evaluated over the post-update database state σ_1 . These quads become valid after the update and must be inserted into the view.

Step 2: Computing the relation-relevant contribution (Lines 19–39).

If Ψ is relation-relevant to R , the update affects the evaluation of Ψ indirectly. In this case, tuples in the pivot relation $R^* = \text{pivot}(\Psi)$ whose derivations depend on tuples in D or I may have their

Algorithm 1: COMPUTE_DELTA_R(W, u, R, σ_1, W_0)**Input:** RDB2RDF view $W = (V, S, M)$; update $u = (D, I)$ over relation R ; post-update database state σ_1 ; pre-update RDF state W_0 .**Output:** Changeset $\langle \Delta^-(u), \Delta^+(u) \rangle$.

```

1.  $\Delta^-(u) \leftarrow \emptyset$ ;
2.  $\Delta^+(u) \leftarrow \emptyset$ ;
3.  $Relev(R) \leftarrow \{\Psi \in M \mid \Psi \text{ is pivot-relevant to } R \vee \Psi \text{ is relation-relevant to } R\}$ ;
4. foreach  $\Psi \in Relev(R)$  do
5.    $\Delta_{pivot}^-[ \Psi ](u) \leftarrow \emptyset$ ;
6.    $\Delta_{rel}^-[ \Psi ](u) \leftarrow \emptyset$ ;
7.    $\Delta_{pivot}^+[ \Psi ](u) \leftarrow \emptyset$ ;
8.    $\Delta_{rel}^+[ \Psi ](u) \leftarrow \emptyset$ ;
  // 1. Pivot-relevant contribution
9.   if  $\Psi$  is pivot-relevant to updates on  $R$  then
10.    foreach  $d \in D$  do
11.       $\Delta_{pivot}^-[ \Psi ](d) \leftarrow \{(s, p, o, g) \in W_0 \mid s = URI(d) \wedge g = URI(\Psi)\}$ ;
12.       $\Delta_{pivot}^-[ \Psi ](u) \leftarrow \Delta_{pivot}^-[ \Psi ](u) \cup \Delta_{pivot}^-[ \Psi ](d)$ ;
13.    end
14.    foreach  $i \in I$  do
15.       $\Delta_{pivot}^+[ \Psi ](i) \leftarrow \{(s, p, o, g) \mid (s, p, o) \in \Psi[r \leftarrow i](\sigma_1) \wedge g = URI(\Psi)\}$ ;
16.       $\Delta_{pivot}^+[ \Psi ](u) \leftarrow \Delta_{pivot}^+[ \Psi ](u) \cup \Delta_{pivot}^+[ \Psi ](i)$ ;
17.    end
18.  end
  // 2. Relation-relevant contribution
19.  if  $\Psi$  is relation-relevant to updates on  $R$  then
20.     $R^* \leftarrow pivot(\Psi)$ ;
    // 2.1 Affected pivot tuples for deletion
21.    if  $R$  occurs only once in  $path(\Psi)$  then
22.       $A_{\Psi}^- \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_R \leftarrow d](\sigma_1)\}$ ;
23.    end
24.    else
25.       $\sigma_R^0 \leftarrow (R(\sigma_1) \cup D) \setminus I$ ;
26.       $A_{\Psi}^- \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists d \in D, \exists r_k \in Var_R(\Psi), \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow d](\sigma_1[R \leftarrow \sigma_R^0])\}$ ;
27.    end
28.    foreach  $p^* \in A_{\Psi}^-$  do
29.       $S_1[\Psi](p^*) \leftarrow \{(s, p, o, g) \in W_0 \mid s = URI(p^*) \wedge g = URI(\Psi)\}$ ;
30.       $S_2[\Psi](p^*) \leftarrow \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1) \wedge g = URI(\Psi)\}$ ;
31.       $\Delta_{rel}^-[ \Psi ](p^*) \leftarrow S_1[\Psi](p^*) \setminus S_2[\Psi](p^*)$ ;
32.       $\Delta_{rel}^-[ \Psi ](u) \leftarrow \Delta_{rel}^-[ \Psi ](u) \cup \Delta_{rel}^-[ \Psi ](p^*)$ ;
33.    end
    // 2.2 Affected pivot tuples for insertion
34.     $A_{\Psi}^+ \leftarrow \{p^* \in R^*(\sigma_1) \mid \exists i \in I, \exists r_k \in Var_R(\Psi), \exists (s, p, o) \in \Psi[r^* \leftarrow p^*, r_k \leftarrow i](\sigma_1)\}$ ;
35.    foreach  $p^* \in A_{\Psi}^+$  do
36.       $\Delta_{rel}^+[ \Psi ](p^*) \leftarrow \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1) \wedge g = URI(\Psi)\}$ ;
37.       $\Delta_{rel}^+[ \Psi ](u) \leftarrow \Delta_{rel}^+[ \Psi ](u) \cup \Delta_{rel}^+[ \Psi ](p^*)$ ;
38.    end
39.  end
  // 3. Overall deletion and insertion contributions
40.   $\Delta^-[ \Psi ](u) \leftarrow \Delta_{pivot}^-[ \Psi ](u) \cup \Delta_{rel}^-[ \Psi ](u)$ ;
41.   $\Delta^+[ \Psi ](u) \leftarrow \Delta_{pivot}^+[ \Psi ](u) \cup \Delta_{rel}^+[ \Psi ](u)$ ;
42.   $\Delta^-(u) \leftarrow \Delta^-(u) \cup \Delta^-[ \Psi ](u)$ ;
43.   $\Delta^+(u) \leftarrow \Delta^+(u) \cup \Delta^+[ \Psi ](u)$ ;
44. end
45. return  $\langle \Delta^-(u), \Delta^+(u) \rangle$ ;

```

RDF contributions modified. The computation proceeds in two steps.

Step 2.1: Affected pivot tuples for deletion (Lines 21–33).

Compute A_{Ψ}^- , the sets of pivot tuples in R^* affected by the deleted tuples in D . Two cases must be distinguished:

- **Single occurrence of R in $path(\Psi)$ (Lines 21–23):** the affected pivot tuples can be identified directly from the post-update state σ_1 , without simulating the pre-update state of R .
- **Multiple occurrences of R in $path(\Psi)$ (Lines 24–27):** the computation simulates the pre-update state of R by binding, in Ψ , occurrences of tuple variables associated with R to tuples in the post-update state of R or in D , but not in I (Lines 25 and 26), while binding the occurrences of any other relation S to tuples in the post-update state of S .

Intuitively, the affected sets contain the pivot tuples whose RDF derivations depend on tuples modified by the update.

For each affected pivot tuple $p^* \in A_{\Psi}^-$, define:

- $S_1[\Psi](p^*)$: quads created by Ψ from p^* in the pre-update RDF state W_0 .
- $S_2[\Psi](p^*)$: quads created by Ψ from p^* in the post-update database state σ_1 .

The deletion contribution associated with p^* is then:

$$\Delta_{rel}^-[\Psi](p^*) = S_1[\Psi](p^*) \setminus S_2[\Psi](p^*)$$

Intuitively, the set difference removes from the view exactly the quads that were valid before the update but are no longer generated after the update.

Step 2.2: Affected pivot tuples for insertion (Lines 34–38).

Symmetrically, for each affected pivot tuple $p^* \in A_{\Psi}^+$, the insertion contribution is defined as the set of quads generated by Ψ from p^* in the post-update state σ_1 :

$$\Delta_{rel}^+[\Psi](p^*) = \{(s, p, o, g) \mid (s, p, o) \in \Psi[r^* \leftarrow p^*](\sigma_1), g = URI(\Psi)\}$$

Step 3: Overall deletion and insertion contributions (Lines 40–43).

The overall deletion and insertion contributions of Ψ are then obtained from the combined pivot and relation-relevant contributions:

$$\Delta^-[\Psi](u) = \Delta_{pivot}^-[\Psi](u) \cup \Delta_{rel}^-[\Psi](u)$$

$$\Delta^+[\Psi](u) = \Delta_{pivot}^+[\Psi](u) \cup \Delta_{rel}^+[\Psi](u)$$

This concludes the brief remarks on Algorithm 1.

4.3 Correctness and Minimality

This section presents a series of results showing that the incremental formulation of changeset computation is correct. Specifically, the results show that applying the incremental changeset to the pre-update RDF state yields exactly the RDF state of the view after the update. The section also states that the changesets satisfy a minimality criterion.

Due to space limitations, this section only contains very brief outlines of the proofs. The reader is referred to the supplemental material for detailed proofs.

In what follows, let $\mathcal{W} = (V, S, M)$ be an RDB2RDF view, $u = (D, I)$ be an update on a relation R , σ_0 and σ_1 be the database

states before and after applying u , and $W_0 = RDF_State(\mathcal{W}, \sigma_0)$ and $W_1 = RDF_State(\mathcal{W}, \sigma_1)$ be the view states before and after applying u . Let $\Delta^-(u)$ and $\Delta^+(u)$ be the deletion and insertion components defined as in Section 4.2.

Let σ be a relational database state. Recall that the RDF state of $\mathcal{W}$ under σ is defined as

$$RDF_State(\mathcal{W}, \sigma) = \bigcup_{\Psi \in M} \bigcup_{t \in pivot(\Psi)(\sigma)} RDF_State[\Psi](t, \sigma)$$

The first lemma shows that each quad in an RDF view state has a unique derivation associated with a single pivot tuple and transformation rule.

LEMMA 4.1 (UNIQUENESS OF QUAD GENERATION). *For every quad $(s, p, o, g) \in RDF_State(\mathcal{W}, \sigma)$, there exists a unique transformation rule $\Psi \in M$ and a unique pivot tuple $p^* \in pivot(\Psi)(\sigma)$ such that*

$$(s, p, o, g) \in RDF_State[\Psi](p^*, \sigma)$$

Brief Proof Outline. By definition of $RDF_State(\mathcal{W}, \sigma)$, every quad (s, p, o, g) is generated by applying some transformation rule $\Psi \in M$ to some pivot tuple $p^* \in pivot(\Psi)(\sigma)$. Thus, existence is immediate. To prove uniqueness, assume that a quad (s, p, o, g) is generated by two pairs (Ψ_1, p_1) and (Ψ_2, p_2) . Using an argument by contradiction, one can prove that $\Psi_1 = \Psi_2$ and $p_1 = p_2$. $\square$

The next two lemmas show that the incremental deletion and the incremental insertion components capture exactly the quads whose derivations are invalidated by deletions, or created by insertions.

LEMMA 4.2 (CHARACTERIZATION OF THE DELETION COMPONENT). *For an update $u = (D, I)$, the view states W_1 and W_0 , and the deletion and insertion components $\Delta^-(u)$ and $\Delta^+(u)$ are such that*

$$(a) \text{ (For any quad } q)((q \in W_0 \wedge q \notin \Delta^-(u)) \Rightarrow q \in W_1).$$

$$(b) \text{ (For any quad } q)(q \in W_1 \Rightarrow (q \notin \Delta^-(u) \vee q \in \Delta^+(u))).$$

Brief Proof Outline. The proof of each part follows a case analysis much in the same way as in Section 4.2.1 by considering deleted quads that are generated by tuples that are pivot-relevant or relation-relevant to a TR Ψ . The proof uses Lemma 4.1. $\square$

A quad q has a derivation under state σ iff there is a rule Ψ , with pivot relation T^* , and a pivot tuple $t^* \in T^*(\sigma)$ such that $q \in \Psi[r \leftarrow t^*](\sigma)$.

LEMMA 4.3 (CHARACTERIZATION OF THE INSERTION COMPONENT). *For an update $u = (D, I)$, the insertion component $\Delta^+(u)$ is the set of quads whose derivation becomes valid in σ_1 due to some tuple $i \in I$.*

Brief Proof Outline. Albeit much simpler, the proof again follows a case analysis by considering inserted quads that are generated by tuples that are pivot-relevant or relation-relevant to a TR Ψ . Again, the proof uses Lemma 4.1. $\square$

Theorem 4.4 states that the insertion and deletion changesets are correct.

THEOREM 4.4 (CORRECTNESS OF THE INCREMENTAL CHANGESETS). *For an update $u = (D, I)$, the view states W_1 and W_0 , and the deletion and insertion components $\Delta^-(u)$ and $\Delta^+(u)$ are such that*

$$W_1 = (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$$

Brief Proof Outline. The proof follows from Lemmas 4.2 and 4.3.□

Finally, Theorem 4.4 has a simple corollary. Let σ_D be the database state after applying the deletions in D , but before applying the insertions in I . Let $W_D = \text{RDF_State}(\mathcal{W}, \sigma_D)$ be the view state corresponding to σ_D .

COROLLARY 4.5 (MINIMALITY OF THE INCREMENTAL CHANGESSETS). *For an update $u = (D, I)$, the view states W_1 , W_D , and W_0 , and the deletion and insertion components $\Delta^-(u)$ and $\Delta^+(u)$ are such that*

- (a) $\Delta^-(u) = (W_0 \setminus W_D)$
- (b) $(\Delta^+(u) \setminus W_D) = (W_1 \setminus W_D)$
- (c) *If $(\Delta^+(u) \cap W_D) = \emptyset$, then $\Delta^+(u) = (W_1 \setminus W_D)$*

Brief Proof Outline. The proof follows from Theorem 4.4.□

Corollary 4.5 suggests that the changesets are minimal in the following sense. Part (a) says that $\Delta^-(u)$ is exactly the set of quads that are removed from the initial view state W_0 as a result of the deletions in the update u (but before the insertions). Likewise, Part (b) states that $\Delta^+(u)$ is the set of quads that are inserted into the intermediate view state W_D , created after the deletions, as a result of the insertions in the update u . But we have to ignore quads that are already in W_D , since we do not rule out stale insertions that lead to duplicates. Part (c) is a simple consequence of Part (b).

5 Operationalization of the Formal Model

5.1 Overview

A natural question arising from the previous sections is whether the formal model can be implemented in practice. To answer this question, this section introduces an LLM-based prototype translation workflow that starts with R2RML mappings that define an entity-preserving RDB2RDF view and produces the required AFTER triggers and related artifacts to maintain the view incrementally.

Section 5.2 introduces an asynchronous architecture that re-engineers Algorithm 1 to separate access to the post-update database state σ_1 from access to the pre-update view state W_0 , since the database and the view are typically in distinct environments. Section 5.3 then describes the LLM-based prototype translation workflow that produces the required artifacts to maintain the view incrementally.

It should be stressed that the goal of this section is not to introduce a production-ready compiler or a new LLM code-generation method. Instead, the section uses an LLM as an experimental instrument to investigate whether the formal model is sufficiently precise and constructive to guide the generation of executable incremental-maintenance logic. The LLM is not used to establish the correctness of the proposed framework, which follows from the formal results.

5.2 An Asynchronous Architecture

The asynchronous architecture, shown in Figure 2, combines AFTER triggers with an asynchronous maintenance worker.

Instead of directly computing and applying the changeset inside a relational transaction, a trigger captures a snapshot of the update event on a relation R (the transition tables D and I), and computes the affected tuples for deletion $A^-[\Psi](u)$, the post-update contributions $S_2[\Psi](u)$ to $\Delta^-(u)$, and the insertion changeset $\Delta^+(u)$ (which requires computing the affected tuples

for insertion $A^+[\Psi](u)$). This information is then stored in a maintenance queue. The AFTER trigger for a relation R calls the function `COMPUTE_CHANGESSET_R`, shown in Algorithm 2, to compute $A^-[\Psi](u)$, $S_2[\Psi](u)$, and $\Delta^+[\Psi](u)$ and update the queue.

An external worker processes queued events sequentially, preserving the order of update commits. For each event, the worker first queries the triplestore to retrieve from the view state W_0 the corresponding deletion contribution of deleted tuples, $\Delta_{pivot}^-[\Psi](u)$, and the pre-update RDF contributions, $S_1[\Psi](u)$. Then, it computes the deletion contribution using the set difference $(S_1[\Psi](u) \setminus S_2[\Psi](u))$, applies $\Delta^-(u)$ to the view, and then applies $\Delta^+(u)$.

This architecture offers several advantages. First, it avoids executing SPARQL queries inside relational transactions, preventing triplestore latency or temporary unavailability from blocking database updates. Second, it stores the relational context of the update at commit time, ensuring that subsequent database updates do not affect the computation of the changeset. Third, the queue-based design improves recoverability, as maintenance events can be retried or processed asynchronously without affecting transactional workloads.

5.3 An LLM-based Prototype Translation Workflow

The translation workflow implements a two-phase process. Each phase is implemented by a specialized LLM-based module that generates a well-defined intermediate artifact. Since LLMs may occasionally produce incomplete or inconsistent outputs, each generated artifact is independently validated before being accepted as the input to the subsequent phase. This validation process ensures that the output of each phase conforms to the formal model.

Phase 1 – Translating R2RML Statements into Transformation Rules

The first phase translates an entity-preserving R2RML mapping into a validated set of transformation rules. An LLM-assisted transformation rules translation module implements this phase, guided by a prompt (available in the supplemental material) whose inputs are the relational database schema and the R2RML mapping.

This module performs the following tasks:

- (1) Identifies pivot relations and verifies entity preservation.
- (2) Derives URI constructors and the corresponding hasURI predicates.
- (3) Validates the pivot relations and URI constructors.
- (4) Generates and validates the transformation rules.

An LLM performs the validations in Steps 3 and 4 automatically. If some URI or transformation rule fails validation, it is flagged for subsequent manual evaluation.

The output is a structured JSON specification representing a validated intermediate artifact. R2RML mappings that fail validation are reported together with the detected inconsistencies and marked as “Manual Analysis Required”. No transformation rules are generated for invalid mappings.

This intermediate representation completely separates semantic reasoning from code generation. Consequently, the second

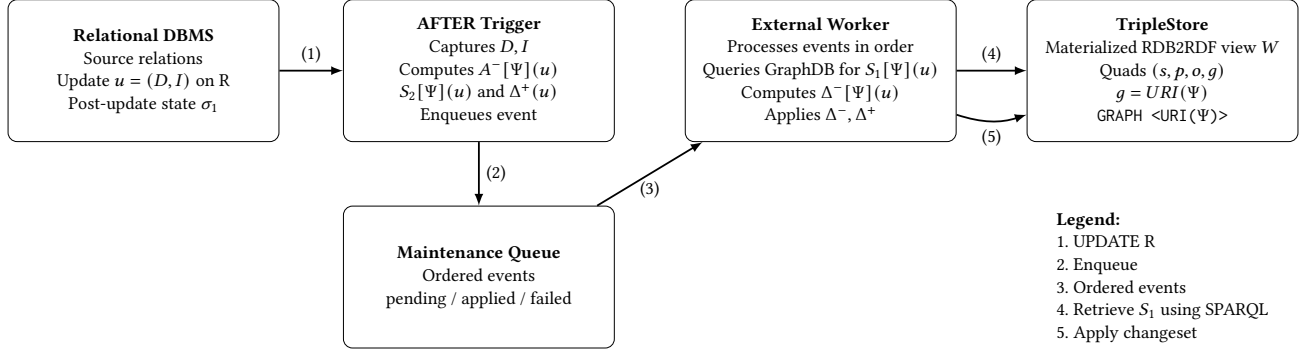


Figure 2: Asynchronous architecture for incremental maintenance of a materialized RDB2RDF view.

Algorithm 2: Function COMPUTE_CHANGESET_R for the Asynchronous Architecture

Input: RDB2RDF view $W = (V, S, M)$; update $u = (D, I)$ over relation R .

Output: Insertions on tables *rdf_maintenance_queue* and *rdf_rule_contribution*.

// Event-level queue initialization:

1. Insert one row into *rdf_maintenance_queue* for the statement-level update $u = (D, I)$;
- // Rule-level computation and publication:
2. **foreach** $\Psi \in \text{Relev}(R)$ **do**
 - // Execute the following two steps sequentially:
 - // STEP 1 – Incremental Maintenance Computation for Ψ
 - 3. $\Delta_{pivot}^+[\Psi](u) \leftarrow \emptyset$;
 - 4. $\Delta_{rel}^+[\Psi](u) \leftarrow \emptyset$;
 - 5. **if** Ψ is pivot-relevant to updates on R **then**
 - 6. | Compute $\Delta_{pivot}^+[\Psi](u)$;
 - 7. **end**
 - 8. **if** Ψ is relation-relevant to updates on R **then**
 - 9. | Compute $A^-[\Psi](u)$, $A^+[\Psi](u)$, $S_2[\Psi](u)$, and $\Delta_{rel}^+[\Psi](u)$;
 - 10. **end**
 - 11. $\Delta^+[\Psi](u) \leftarrow \Delta_{pivot}^+[\Psi](u) \cup \Delta_{rel}^+[\Psi](u)$;
 - // STEP 2 – Rule Contribution Publication
 - 12. Insert one row into *rdf_rule_contribution* for Ψ with $A^-[\Psi](u)$, $S_2[\Psi](u)$, and $\Delta^+[\Psi](u)$
 - 13. | linked to the *event_id* created in *rdf_maintenance_queue*;
14. **end**

translation phase no longer needs to interpret the R2RML mapping and instead relies exclusively on the validated transformation rules.

Phase 2 – Translating Transformation Rules into AFTER Triggers

The second phase translates the transformation rules generated during Phase 1 into AFTER triggers, as discussed in Section 5.2. An LLM-assisted trigger translation module implements this phase, guided by a prompt (available in the supplemental material), whose inputs consist of the transformation rules generated during Phase 1, and the incremental maintenance framework of Section 5.2.

Each trigger captures a snapshot of the update event on a relation R (the transition tables D and I), and computes the affected tuples for deletion $A^-[\Psi](u)$, the post-update contributions $S_2[\Psi](u)$ to $\Delta^-(u)$, and the insertion changeset $\Delta^+(u)$ (which requires computing the affected tuples for insertion $A^+[\Psi](u)$).

The resulting code operates on the relational database and communicates with the asynchronous maintenance worker through the queue, without directly accessing the materialized RDF view.

5.4 Remarks on Concurrent Updates

Very briefly, a simple strategy to accommodate (single relation) concurrent updates in the architecture described in Section 5.2 would be as follows:

- (1) The function COMPUTE_CHANGESET_R, shown in Algorithm 2, must be run in the same transaction as the update; that is, the update and all corresponding triggers must run in the same transaction.
- (2) The external worker must serialize the updates to the view, in transaction commit order; that is, it must process all queued events generated by an update sequentially, before processing the queued events generated by the next update, in transaction commit order.

6 Operational Feasibility Study

The goal of this section is to investigate the operationalization of the formal model proposed in Section 5. As a case study, it adopts the LinkedBrainz RDB2RDF View, defined on top of the MusicBrainz relational database using a set of R2RML statements. It validates the complete cycle from R2RML statements to change-set computation.

This section focuses on correctness issues. Therefore, its goal is specialized into the following questions:

RQ1. Correctness of the Transformation Rules Translation: Can R2RML statements be correctly translated into transformation rules?

RQ2. Correctness of the AFTER Triggers Translation: Can transformation rules be correctly translated into executable AFTER triggers and the maintenance worker?

RQ3. End-to-End Correctness: Do the changesets specified by the transformation rules produce the same RDF state as a complete recomputation?

RQ4. Human Intervention: Which parts of the operationalization require human decisions, and why?

The supplemental material contains the complete case study.

6.1 Setup

The feasibility study was based on:

- *Relational Database*: a local copy of the *MusicBrainz* database, stored in PostgreSQL.
- *LinkedBrainz RDB2RDF View*: an RDB2RDF view of the *MusicBrainz* database, materialized in GraphDB.
- *Selected relations*: the relations *ARTIST*, *MEDIUM*, and *TRACK*, chosen because they occur in complex transformation rules.
- *Selected updates*: one insertion, one deletion, and one update per selected relation, with the expected changesets.
- *Selected LLMs*: gpt-5.6-sol (temperature: 1.0, reasoning_effort = "medium") and qwen3.8-max-0902 (temperature: 1.0, reasoning = "medium"), referred to simply as GPT-5.6 and Qwen3.8 in what follows.

MusicBrainz³ has 374 tables, with data about artists, release groups, etc. The LinkedBrainz RDB2RDF View uses the Music Ontology vocabulary and is specified in R2RML by 57 TriplesMaps⁴.

6.2 Correctness Validation Procedure

Transformation Rules Translation Correctness. This step addresses RQ1 and, in part, RQ4. GPT-5.6 was prompted to translate the R2RML statements of the case study into transformation rules. Then, the results were verified, separating the validation of translation correctness for the URI constructor from the transformation rules. GPT-5.6 also helped in iteratively refining the translation prompt until correct, traceable transformation rules were produced.

URI constructor correctness validation checks the preservation of the complete subject template, placeholder expressions, and ordering, as well as encoding semantics and consistency across mappings that represent the same semantic entity. Transformation rules translation correctness validation checks the RDF class or property, RDF term type, selection conditions, value transformations, null handling, and relational paths.

³Available at <https://musicbrainz.org>

⁴Available at <https://docs.openlinksw.com/virtuoso/rdfviewsbusintmbr/>

When the R2RML input contains ambiguities, conflicting URI templates, malformed SQL expressions, unresolved relational paths, or potentially invalid URI values, the translation phase still generates a candidate rule and marks it for human validation. The output includes the supporting evidence and the issue requiring review. A candidate is validated only after passing all automatic checks or after the corresponding human decision is explicitly recorded.

AFTER Trigger Translation Correctness. This step addresses RQ2 and, in part, RQ4. GPT-5.6 and Qwen3.8 were prompted to translate the transformation rules for the selected relations – *ARTIST*, *MEDIUM*, and *TRACK* – into AFTER Triggers.

For each combination of LLM and selected relation, the correctness of the generated trigger code was manually validated according to the criteria presented in Table 3.

End-to-End Correctness. This step addresses RQ3. The AFTER triggers for the selected relations were executed using different update scenarios. To be more informative, correctness validation was subdivided into two tasks.

Rule Contribution Correctness. Rule contribution correctness validation verifies whether the AFTER triggers compute the expected maintenance data for each relevant transformation rule and update scenario. In more detail, for each update u and each relevant transformation rule $\Psi \in \text{Relev}(R)$, the generated rule contributions

$$A^-[\Psi], A^+[\Psi], S2[\Psi], \Delta_{pivot}^+[\Psi], \Delta_{rel}^+[\Psi], \Delta^+[\Psi]$$

were manually compared against the expected results derived from the incremental maintenance semantics of Section 4.

Changeset Correctness. This step addresses RQ3 by providing end-to-end validation of the correctness of the generated maintenance code. It verifies whether the generated changesets produce the same RDF view as a full recomputation, which is independently obtained.

Let W_0 denote the RDF view before the update and $W_1 = M(\sigma_1)$ the RDF view obtained by applying the complete RDB2RDF mapping to the updated database state σ_1 . The generated triggers are considered correct iff the RDF view reconstructed from their computed changesets satisfies $W_1 = (W_0 \setminus \Delta^-(u)) \cup \Delta^+(u)$.

Table 3: Trigger Translation and End-to-End Correctness.

Criteria	Description
<i>AFTER Trigger Translation Correctness</i>	
Transformation Rule Identification	Does the LLM-assisted trigger translation module correctly identify the relevant transformation rules?
Rule Contribution Computation	Does the LLM-assisted trigger translation module correctly generate the code for the rule contributions?
Code Minimality	Does the LLM-assisted trigger translation module generate code which is concise and free of unnecessary operations?
Hallucination	Does the LLM-assisted trigger translation module introduce unsupported assumptions or irrelevant joins?
<i>End-to-End Correctness</i>	
Rule Contribution Correctness	Do the generated triggers compute the expected rule contributions for each relevant transformation rule and update scenario?
Changeset Correctness	Do the generated triggers compute the expected changesets for each update scenario?

6.3 Results

This section summarizes the major results obtained. The reader is referred to the supplemental material for a detailed description of the results, backed up by the evaluation logs.

Transformation Rules Translation Correctness. All R2RML statements of the case study were classified as entity-preserving and translated into 89 transformation rules. The correctness of 78 transformation rules was automatically validated, but 11 (12.4%) required human validation, primarily due to malformed SQL joins or mapping inconsistencies.

AFTER Trigger Translation and End-to-End Correctness. Table 4 summarizes the AFTER trigger translation and end-to-end correctness validation for the TRACK relation, with the triggers obtained with GPT-5.6 and Qwen3.8. Tables 6 and 5 show the results for ARTIST and MEDIUM obtained with GPT-5.6 only. The models were run at least three times to obtain stable results.

Summary. To summarize, the results suggest that:

- GPT-5.6 correctly translated all R2RML statements into transformation rules, but some rules required human validation.
- GPT-5.6 generated correct and optimized AFTER trigger code in most cases.
- Qwen 3.8-Max-Preview generated correct and concise AFTER trigger code in some cases.

Table 4: Trigger Translation and End-to-End Correctness Validation – Results for Relation Track – GPT-5.6 and Qwen3.8.

Criteria	Model	Result
<i>AFTER Trigger Translation Correctness</i>		
Transformation rule Identification	GPT	Identified all relevant rules.
	Qwen	Identified all relevant rules.
Rule Contribution Computation	GPT	Correctly computed all rule contributions.
	Qwen	Correctly computed all rule contributions.
Code Minimality	GPT	Produced a redundant intermediate code.
	Qwen	Produced a redundant intermediate code.
Hallucination	GPT	Did not hallucinate.
	Qwen	Did not hallucinate.
<i>End-to-End Correctness</i>		
Rule Contribution Correctness	GPT	The triggers generated contributions for all 7 rules evaluated. The generated quadruples are correct for 5 of the rules, while the remaining 2 rules show discrepancies regarding literal datatypes.
	Qwen	Of the contributions generated by the 7 rules, 4 produced correct quadruples. A fifth rule was considered correct with the inclusion of <i>xsd:string</i> . The other 2 differ only in the data types of the literals
Changeset Correctness	GPT	The computed changesets correspond exactly to the expected changesets.
	Qwen	(Failed to execute)

Table 5: Trigger Translation and End-to-End Correctness Validation – Results for Relation Artist – GPT-5.6.

Criteria	Model	Result
<i>AFTER Trigger Translation Correctness</i>		
Transformation rule Identification	GPT	identified all relevant rules
Computation of Rules Contributions	GPT	correctly produced all relational joins
Code Minimality	GPT	relatively clean and concise
Hallucinations	GPT	did not hallucinate
<i>End-to-End Correctness</i>		
Rule Contribution Correctness	GPT	Out of the 14 rules, the quadruples for 13 of them were generated correctly. The 14th rule generated quadruples with a URI encoding error due to a whitespace character.
Changeset Correctness	GPT	the computed changesets correspond exactly to the expected changesets

Table 6: Trigger Translation and End-to-End Correctness Validation – Results for Relation Medium – GPT-5.6.

Criteria	Model	Result
<i>AFTER Trigger Translation Correctness</i>		
Transformation rule Identification	GPT	identified all relevant rules
Computation of Rules Contributions	GPT	correctly produced all relational joins
Code Minimality	GPT	produced minimal code
Hallucinations	GPT	did not hallucinate
<i>End-to-End Correctness</i>		
Rule Contribution Correctness	GPT	The trigger on medium relation generated contributions for all 6 rules evaluated. The produced quadruples are correct for all of the rules
Changeset Correctness	GPT	the computed changesets correspond exactly to the expected changesets

7 Conclusions

The first contribution of this paper was a formal model of the deletion and insertion components of RDB2RDF changesets. Existing formulations of incremental RDB2RDF maintenance depend on the pre-update relational state when computing deletion changesets. This creates a mismatch with statement-level AFTER triggers, which execute when the database is already in its post-update state and have direct access only to the transition tables and the current database state. This paper introduces a formal model that removes this mismatch through a post-update formulation and proves the correctness of the resulting changesets.

The second contribution is to propose an operational architecture and implement a prototype for incremental maintenance of RDB2RDF views that realizes the formal model. It starts with R2RML statements that define an entity-preserving RDB2RDF view and produces the required AFTER triggers and related artifacts to maintain the view incrementally. The prototype provides evidence that the formal model is sufficiently constructive to support generating maintenance triggers from the RDB2RDF view mapping specification.

The third contribution was to conduct a feasibility study of the operational architecture, based on an RDB2RDF view defined by R2RML statements. It illustrates that an LLM, guided by appropriate prompts and accessing the formal model, can generate code to compute and publish the changesets.

As future work, the LLM-based prototype will be re-engineered as a multi-agent system with human-in-the-loop to facilitate compiling R2RML statements into AFTER triggers and installing them in the source relational database.

Acknowledgments

This work was partly funded by FAPERJ under grant E-26/204.322/2024 and by CNPq under grant 306113/2026-0.

Declaration of use of Generative AI

In Sections 4.2 and 5, ChatGPT was used to help check the definitions and define the architecture. In Section 6, GPT-5.6 and Qwen3.8 were utilized to generate code.

Artifacts

The extended paper, with the complete formal definitions, results, and proofs, and the code, prompts, database, and RDB2RDF view adopted in the case study are available at <https://github.com/semantic-ekgraphs/IVM.git>

References

- [1] Mahmoud Abo Khamis, Eden Chmielewski, Andrei Draghici, Ahmet Kara, and Dan Olteanu. 2026. Maintaining Queries under Updates Using Heavy-Light Partitioning of the Input Relations. *Proc. ACM Manag. Data* 4, 2, Article 109 (May 2026), 22 pages. doi:10.1145/3801905
- [2] M. Akhtar Ali, Alvaro A. A. Fernandes, and Norman W. Paton. 2003. MOVIE: an incremental maintenance system for materialized object views. *Data Knowl. Eng.* 47, 2 (Nov. 2003), 131–166. doi:10.1016/S0169-023X(03)00048-X
- [3] Cristian Augusto, Antonia Bertolino, Guglielmo De Angelis, Francesca Lonetti, and Jesús Morán. 2025. Large Language Models for Software Testing: A Research Roadmap. doi:10.48550/arXiv.2509.25043 arXiv:2509.25043 [cs.SE]
- [4] Ilaria Battiston, Kriti Kathuria, and Peter Boncz. 2024. OpenIVM: a SQL-to-SQL Compiler for Incremental Computations. In *Companion of the 2024 International Conference on Management of Data* (Santiago AA, Chile) (*SIGMOD '24*). Association for Computing Machinery, New York, NY, USA, 516–519. doi:10.1145/3626246.3654743
- [5] Arshad Beg, Diarmuid O'Donoghue, and Rosemary Monahan. 2025. Leveraging LLMs for Formal Software Requirements – Challenges and Prospects. arXiv:2507.14330 [cs.SE] <https://arxiv.org/abs/2507.14330>
- [6] Mihai Budiú, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen. 2023. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *Proc. VLDB Endow.* 16, 7 (March 2023), 1601–1614. doi:10.14778/3587136.3587137
- [7] Stefano Ceri and Jennifer Widom. 1991. Deriving Production Rules for Incremental View Maintenance. In *Proceedings of the 17th International Conference on Very Large Data Bases (VLDB '91)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 577–589.
- [8] Alessio Ferrari and Paola Spoletini. 2025. Formal requirements engineering and large language models: A two-way roadmap. *Information and Software Technology* 181 (2025), 107697. doi:10.1016/j.infsof.2025.107697
- [9] Soonbo Han and Zachary G. Ives. 2025. Implementing Views for Property Graphs. *SIGMOD Rec.* 54, 1 (2025), 59–68. doi:10.1145/3733620.3733633
- [10] Xiao Hu and Qichen Wang. 2025. Towards Update-Dependent Analysis of Query Maintenance. *Proc. ACM Manag. Data* 3, 2, Article 117 (2025), 25 pages. doi:10.1145/3725254
- [11] Xueyun Jin and Husheng Liao. 2010. An Algorithm for Incremental Maintenance of Materialized XPath View. In *Web-Age Information Management*, Lei Chen, Changjie Tang, Jun Yang, and Yunjun Gao (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 513–524.
- [12] Pengyuan Li, Boris Glavic, Dieter Gawlick, Vasudha Krishnaswamy, Zhen Hua Liu, Danica Porobic, and Xing Niu. 2026. In-memory Incremental Maintenance of Provenance Sketches. In *Proceedings 29th International Conference on Extending Database Technology, EDBT 2026, Tampere, Finland, March 24-27, 2026*. OpenProceedings.org, 56–68. doi:10.48786/EDBT.2026.05
- [13] Roman Lukyanenko et al. 2025. Manifesto for Software Development and Modeling in the Age of Artificial Intelligence. *SSRN Electronic Journal* (12 2025). doi:10.2139/ssrn.5881104
- [14] Dan Olteanu. 2024. Recent Increments in Incremental View Maintenance. In *Companion of the 43rd Symposium on Principles of Database Systems* (Santiago AA, Chile) (*PODS '24*). Association for Computing Machinery, New York, NY, USA, 8–17. doi:10.1145/3635138.3654763
- [15] Dore Staquet, Bart Buelens, and Jan Van den Bussche. 2026. Incremental View Maintenance for SPARQL Queries: Adapting the Counting Algorithm. *ACM Trans. Web* 20, 4, Article 47 (Sept. 2026), 34 pages. doi:10.1145/3796549
- [16] Christoforos Svingos, Andre Hernich, Hinnerk Gildhoff, Yannis Papakonstantinou, and Yannis Ioannidis. 2023. Foreign Keys Open the Door for Faster Incremental View Maintenance. *Proc. ACM Manag. Data* 1, 1, Article 40 (May 2023), 25 pages. doi:10.1145/3588720
- [17] Maarten Vandenbrande, Ruben Taelman, Pieter Bonte, and Femke Ongenae. 2025. Incremunica: Web-Based Incremental View Maintenance for SPARQL. In *The Semantic Web: 22nd European Semantic Web Conference, ESWC 2025, Portoroz, Slovenia, June 1–5, 2025, Proceedings, Part II* (Portoroz, Slovenia). Springer-Verlag, Berlin, Heidelberg, 297–315. doi:10.1007/978-3-031-94578-6_17
- [18] Vania Vidal, Narciso Arruda, José Renato da Silva Freitas, and Marco A. Casanova. 2026. Provably Correct Post-Update Incremental Maintenance of RDB2RDF Views (Extended Version). <https://github.com/semantic-ekgraphs/IVM/blob/main/IVM-Extended-Paper.pdf>
- [19] Vania Vidal, Valéria Magalhães Pequeno, Narciso Moura Arruda Júnior, and Marco Antonio Casanova. 2022. Publication and maintenance of RDB2RDF views externally materialized in enterprise knowledge graphs. *International Journal of Web Information Systems* 18, 5-6 (07 2022), 255–285. doi:10.1108/IJWIS-02-2022-0043
- [20] Vania Maria P. Vidal, Marco Antonio Casanova, and Diego Sá Cardoso. 2013. Incremental Maintenance of RDF Views of Relational Data. In *On the Move to Meaningful Internet Systems: OTM 2013 Conferences*. Springer Berlin Heidelberg, Berlin, Heidelberg, 572–587. doi:10.1007/978-3-642-41030-7_42
- [21] Vânia Maria Pontes Vidal, Marco A. Casanova, Luís Eufrazio T. Neto, and José Maria Monteiro. 2014. A semi-automatic approach for generating customized R2RML mappings. In *Proceedings of the 29th Annual ACM Symposium on Applied Computing* (Gyeongju, Republic of Korea) (*SAC '14*). Association for Computing Machinery, New York, NY, USA, 316–322. doi:10.1145/2554850.2554933
- [22] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software Testing With Large Language Models: Survey, Agents, and Vision. *IEEE Trans. Softw. Eng.* 50, 4 (April 2024), 911–936. doi:10.1109/TSE.2024.3368208
- [23] Ritwik Yadav, Supun Abeysinghe, Min Yang, Jeffrey Helt, Manuel Ung, Yuhong Chen, Melody Hu, William Wei, Yiming Yang, Tom van Bussel, Sourav Chatterji, Indrajit Roy, Paul Lappas, Yannis Papakonstantinou, Tahir Fayyaz, Bilal Aslam, Ross Bunker, Michael Armbrust, and Shrikanth Shankar. 2026. Enzyme: Incremental View Maintenance for Data Engineering. In *Companion of the International Conference on Management of Data (India) (SIGMOD Companion '26)*. Association for Computing Machinery, New York, NY, USA, 490–503. doi:10.1145/3788853.3803098
- [24] Yedi Zhang et al. 2025. Position: Trustworthy AI Agents Require the Integration of Large Language Models and Formal Methods. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 82441–82459. <https://proceedings.mlr.press/v267/zhang25ds.html>
- [25] Weijie Zhao, Florin Rusu, Bin Dong, Kesheng Wu, and Peter Nugent. 2017. Incremental View Maintenance over Array Data. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) (*SIGMOD '17*). Association for Computing Machinery, New York, NY, USA, 139–154. doi:10.1145/3035918.3064041